

# Input e output

---

## Input/Output Formattato

*Come formattare l'output in C++ con `setw`, `setprecision` e altre opzioni.*

## Controllare l'aspetto dell'output

Stampare "3.14159265358979" va bene in alcuni casi, ma se stai mostrando un prezzo vuoi "3.14". Se stai costruendo una tabella vuoi che le colonne siano allineate.

Il C++ offre strumenti per controllare con precisione come appaiono i dati sullo schermo. Per usarli, includi

`<iomanip>` :

```
#include <iostream>
#include <iomanip>
using namespace std;
```

## Quante cifre decimali: `setprecision`

Di default, `cout` mostra 6 cifre significative. Con `setprecision` puoi cambiarle:

```
double pi = 3.14159265358979;

cout << pi << endl; // 3.14159 (default: 6 cifre)
cout << setprecision(3) << pi << endl; // 3.14
cout << setprecision(10) << pi << endl; // 3.141592654
```

Per controllare le cifre **dopo la virgola** (non le cifre significative), aggiungi `fixed` :

```
double x = 1234.5678;

cout << fixed << setprecision(2) << x << endl; // 1234.57
cout << fixed << setprecision(4) << x << endl; // 1234.5678
cout << fixed << setprecision(0) << x << endl; // 1235 (arrotondato)
```

Per la notazione scientifica (es. per numeri molto grandi o piccoli):

```
double grande = 1234567.89;
cout << scientific << setprecision(3) << grande << endl; // 1.235e+06
```

## Larghezza minima: `setw`

`setw(n)` riserva almeno `n` spazi per il prossimo valore stampato. Se il valore è più corto, vengono aggiunti spazi davanti (a destra di default):

```
cout << setw(10) << "Nome" << setw(10) << "Età" << endl;
cout << setw(10) << "Alice" << setw(10) << 16 << endl;
cout << setw(10) << "Bob" << setw(10) << 17 << endl;
```

Output:

|       |     |
|-------|-----|
| Nome  | Età |
| Alice | 16  |
| Bob   | 17  |

Attenzione: `setw` è **temporaneo**. Vale solo per il prossimo valore stampato, poi va reimpostato.

## Allineamento: `left` e `right`

Di default i valori vengono allineati a destra. Puoi cambiarlo con `left` :

```
cout << left  << setw(10) << "Alice" << endl; // Alice
cout << right << setw(10) << "Alice" << endl; //      Alice
```

Output:

```
Alice
      Alice
```

`left` e `right` sono **persistenti**: rimangono attivi finché non li cambi.

## Carattere di riempimento: `setfill`

Di default `setw` riempie con spazi. Puoi usare un altro carattere:

```
cout << setfill('*') << setw(10) << "ciao" << endl; // *****ciao
cout << setfill('0') << setw(5) << 42 << endl;      // 00042
```

Utile per creare separatori o codici con zeri iniziali.

## Stampare bool come parole

Di default `true` e `false` vengono stampati come `1` e `0`. Con `boolalpha` vengono stampate le parole:

```
bool b = true;
cout << b << endl;           // 1
cout << boolalpha << b << endl; // true

bool f = false;
cout << boolalpha << f << endl; // false
```

## Riepilogo: persistente vs temporaneo

| Modificatore                                 | Dura fino a...                 |
|----------------------------------------------|--------------------------------|
| <code>fixed</code> , <code>scientific</code> | Finché non lo cambi            |
| <code>left</code> , <code>right</code>       | Finché non lo cambi            |
| <code>boolalpha</code>                       | Finché non lo cambi            |
| <code>setprecision(n)</code>                 | Finché non lo cambi            |
| <code>setfill(c)</code>                      | Finché non lo cambi            |
| <code>setw(n)</code>                         | <b>Solo il prossimo valore</b> |

## Esempio pratico: tabella formattata

```
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

int main() {
    // Intestazione della tabella
    cout << left
         << setw(15) << "Studente"
         << setw(8)  << "Età"
         << setw(10) << "Voto"
         << endl;

    // Riga separatrice
    cout << setfill('-') << setw(33) << "" << endl;
    cout << setfill(' '); // ripristina il riempimento con spazio

    // Dati degli studenti
    struct Studente { string nome; int eta; double voto; };
    Studente studenti[] = {
        {"Alice", 16, 8.5},
        {"Bob", 17, 7.2},
        {"Carlo Verdi", 15, 9.1}
    };

    for (const auto& s : studenti) {
        cout << left
             << setw(15) << s.nome
             << setw(8)  << s.eta
             << fixed << setprecision(1) << setw(10) << s.voto
             << endl;
    }

    return 0;
}
```

Output:

| Studente    | Età | Voto |
|-------------|-----|------|
| Alice       | 16  | 8.5  |
| Bob         | 17  | 7.2  |
| Carlo Verdi | 15  | 9.1  |

## Input con cin

Come leggere dati inseriti dall'utente con cin in C++.

## Come si legge l'input dell'utente?

Un programma che fa sempre la stessa cosa con gli stessi dati non è molto utile. `cin` ti permette di leggere quello che l'utente scrive dalla tastiera, rendendo il programma interattivo.

## Cos'è `cin` ?

`cin` è lo strumento del C++ per leggere l'input dall'utente tramite tastiera. Come `cout`, fa parte della libreria `<iostream>`.

## L'operatore `>>`

L'operatore `>>` "estrae" un valore dall'input e lo salva in una variabile:

```
int x;  
cin >> x;
```

Quando il programma arriva a questa riga, si ferma e aspetta che l'utente scriva qualcosa e prema Invio. Il valore inserito viene salvato nella variabile `x`.

## Leggere un numero intero

```
#include <iostream>
using namespace std;

int main() {
    int eta;
    cout << "Inserisci la tua età: "; // spiega cosa vuoi
    cin >> eta;                       // leggi il valore
    cout << "Hai " << eta << " anni." << endl;
    return 0;
}
```

Esempio di esecuzione:

```
Inserisci la tua età: 16
Hai 16 anni.
```

## Leggere un numero decimale

```
double altezza;
cout << "Inserisci la tua altezza in metri: ";
cin >> altezza;
cout << "Alta/o " << altezza << " m." << endl;
```

## Leggere una parola

Con `cin >>` puoi leggere una parola sola. Il problema: si ferma al primo spazio.

```
string nome;
cout << "Inserisci il tuo nome: ";
cin >> nome;
cout << "Ciao, " << nome << "!" << endl;
```

```
Inserisci il tuo nome: Alice
Ciao, Alice!
```

Se l'utente scrive "Alice Bianchi", `cin >> nome` legge solo "Alice" e scarta "Bianchi".

## Leggere una riga intera (con spazi)

Per leggere tutta la riga compreso gli spazi, usa `getline()` :

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string nome_completo;
    cout << "Inserisci il tuo nome completo: ";
    getline(cin, nome_completo); // legge tutta la riga
    cout << "Ciao, " << nome_completo << "!" << endl;
    return 0;
}
```

```
Inserisci il tuo nome completo: Alice Bianchi
Ciao, Alice Bianchi!
```

## Leggere più valori

Puoi leggere più valori in una volta sola, separati da spazio:

```
int a, b;
cout << "Inserisci due numeri separati da spazio: ";
cin >> a >> b;
cout << "Somma: " << a + b << endl;
```



Inserisci due numeri separati da spazio: 5 3  
Somma: 8

## Attenzione: `cin` e `getline` insieme

Se usi `cin >>` e poi `getline()`, c'è una trappola: quando l'utente preme Invio dopo `cin >>`, il carattere "a capo" rimane nel buffer e viene letto subito da `getline()`, saltando l'input dell'utente.

La soluzione è aggiungere `cin.ignore()` per scartare quel carattere "a capo":

```
int eta;
string nome;

cout << "Età: ";
cin >> eta;

cin.ignore(); // scarta il 'Invio' rimasto nel buffer

cout << "Nome completo: ";
getline(cin, nome); // ora funziona correttamente

cout << nome << " ha " << eta << " anni." << endl;
```

## Esempio pratico

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string nome;
    int eta;
    double altezza;

    cout << "Come ti chiami? ";
    cin >> nome;

    cout << "Quanti anni hai? ";
    cin >> eta;

    cout << "Quanto sei alto/a (in metri)? ";
    cin >> altezza;

    cout << endl;
    cout << "=== I tuoi dati ===" << endl;
    cout << "Nome:    " << nome << endl;
    cout << "Età:      " << eta << " anni" << endl;
    cout << "Altezza: " << altezza << " m" << endl;

    return 0;
}
```

Esempio di esecuzione:

```
Come ti chiami? Alice
Quanti anni hai? 16
Quanto sei alto/a (in metri)? 1.65

=== I tuoi dati ===
Nome:    Alice
Età:     16 anni
Altezza: 1.65 m
```

# Cosa succede se l'utente sbaglia tipo?

Se l'utente scrive del testo dove il programma si aspetta un numero, `cin` entra in uno stato di errore e le letture successive non funzionano. Per ora, puoi supporre che l'utente inserisca sempre il tipo corretto. La gestione degli errori di input è un argomento più avanzato.

---

## Output con cout

*Come visualizzare testo e valori sullo schermo con cout in C++.*

## Come si stampa a schermo in C++?

Un programma che non mostra niente all'utente è inutile. `cout` è il modo principale per comunicare con chi usa il tuo programma: mostrare risultati, messaggi, errori, menu.

È una delle prime cose che impari in C++ e la userai in ogni programma che scrivi.

## Cos'è `cout` ?

`cout` è lo strumento del C++ per stampare testo e valori sullo schermo. Fa parte della libreria `<iostream>` che devi includere all'inizio di ogni programma:

```
#include <iostream>
using namespace std;
```

## L'operatore `<<`

L'operatore `<<` manda qualcosa a `cout`. Pensa a esso come a una freccia: "manda questo testo allo schermo".

```
cout << "Ciao, Mondo!";
```

## Stampa di testo

Metti il testo tra virgolette doppie:

```
cout << "Ciao!";  
cout << "Questa è una frase.";
```

## Andare a capo

Per andare a capo, hai due opzioni:

```
cout << "Prima riga" << endl;    // usando endl  
cout << "Seconda riga\n";        // usando \n nella stringa
```

Entrambe vanno a capo. La differenza è che `endl` è leggermente più lento (svuota il buffer). Per la maggior parte dei programmi, sono equivalenti.

## Stampare più cose nella stessa riga

Puoi concatenare più `<<` per stampare più valori insieme:

```
cout << "Nome: " << "Alice" << endl;  
cout << "Età: " << 16 << endl;  
cout << "Altezza: " << 1.65 << " m" << endl;
```

Output:

```
Nome: Alice  
Età: 16  
Altezza: 1.65 m
```

# Stampare variabili

`cout` funziona con qualsiasi tipo di dato:

```
int x = 42;
double pi = 3.14;
char lettera = 'A';
bool vero = true;

cout << x << endl;           // 42
cout << pi << endl;          // 3.14
cout << lettera << endl;     // A
cout << vero << endl;        // 1 (true viene stampato come 1 di default)
```

# Caratteri speciali nelle stringhe

Puoi inserire caratteri speciali nelle stringhe usando il simbolo `\` :

| Sequenza        | Significato                                    |
|-----------------|------------------------------------------------|
| <code>\n</code> | Nuova riga                                     |
| <code>\t</code> | Tabulazione (tab) — crea uno spazio largo      |
| <code>\\</code> | Il carattere backslash <code>\</code>          |
| <code>\"</code> | Il carattere virgolette doppie <code>"</code>  |
| <code>\'</code> | Il carattere virgolette singole <code>'</code> |

```
cout << "Riga 1\nRiga 2\nRiga 3" << endl;
cout << "Nome:\tAlice" << endl;
cout << "Dice: \"Ciao!\"" << endl;
```

Output:

```
Riga 1
Riga 2
Riga 3
Nome:  Alice
Dice: "Ciao!"
```

## Calcoli direttamente nell'output

Puoi fare calcoli direttamente dentro il `cout` :

```
int a = 5, b = 3;
cout << "Somma: " << a + b << endl;    // 8
cout << "Prodotto: " << a * b << endl;  // 15
```

## Esempio pratico

```
#include <iostream>
using namespace std;

int main() {
    string nome = "Alice";
    int eta = 16;
    double altezza = 1.65;

    cout << "=== Scheda Studente ===" << endl;
    cout << "Nome:    " << nome << endl;
    cout << "Età:      " << eta << " anni" << endl;
    cout << "Altezza: " << altezza << " m" << endl;

    return 0;
}
```

Output:

```
=== Scheda Studente ===  
Nome:    Alice  
Età:     16 anni  
Altezza: 1.65 m
```

## Stringhe in C++

*Come usare le stringhe in C++ con il tipo string.*

## Lavorare con il testo in C++

I programmi lavorano con parole, nomi, frasi, URL, messaggi... non solo numeri. Le **stringhe** sono il modo in cui il programma lavora con il testo.

In C++ moderno si usa il tipo `string`, che rende il lavoro con il testo semplice e sicuro.

## Includere la libreria

```
#include <iostream>  
#include <string>  
using namespace std;
```

## Creare una stringa

```
string nome = "Alice";  
string saluto = "Ciao, mondo!";  
string vuota = ""; // stringa senza nessun carattere
```

Le stringhe usano le **virgolette doppie** `"`. Le virgolette singole `'` sono per i caratteri singoli ( `char` ).

# Unire stringhe: concatenazione

Puoi unire due stringhe con `+` :

```
string nome = "Alice";
string cognome = "Bianchi";
string nomeCompleto = nome + " " + cognome;
cout << nomeCompleto << endl; // Alice Bianchi
```

Per aggiungere qualcosa in coda a una stringa:

```
string frase = "Ciao";
frase += ", mondo";
frase += "!";
cout << frase << endl; // Ciao, mondo!
```

## Lunghezza

Il metodo `.length()` restituisce quanti caratteri ha la stringa:

```
string nome = "Alice";
cout << nome.length() << endl; // 5
```

## Accedere a un singolo carattere

Come gli array, puoi leggere o modificare un singolo carattere usando l'indice tra `[]` . Gli indici partono da 0:



```
string nome = "Alice";
cout << nome[0] << endl; // A - primo carattere
cout << nome[1] << endl; // l
cout << nome[4] << endl; // e - quinto (e ultimo) carattere

// Modifica del primo carattere
nome[0] = 'E';
cout << nome << endl; // Elice
```

## Confrontare stringhe

Puoi confrontare stringhe con gli operatori normali:

```
string a = "ciao";
string b = "ciao";
string c = "hello";

cout << (a == b) << endl; // 1 - sono uguali
cout << (a == c) << endl; // 0 - sono diverse
cout << (a != c) << endl; // 1 - sono diverse
```

Attenzione: il confronto distingue maiuscole e minuscole. `"Ciao"` e `"ciao"` sono considerate diverse.

## Cercare dentro una stringa: `find()`

Il metodo `find()` cerca una parola o un carattere e restituisce la posizione in cui si trova:

```
string frase = "Ciao, mondo!";
int pos = frase.find("mondo");
cout << pos << endl; // 6 - "mondo" inizia alla posizione 6

// Se non trova niente, restituisce string::npos
if (frase.find("xyz") == string::npos) {
    cout << "Non trovato" << endl;
}
```

## Estrarre una parte: `substr()`

Il metodo `substr()` estrae una porzione di stringa:

```
string frase = "Ciao, mondo!";  
// substr(posizione_di_inizio, lunghezza)  
string parte = frase.substr(6, 5);  
cout << parte << endl; // mondo
```

## Convertire numeri in stringhe

Per trasformare un numero in stringa (utile per concatenarlo):

```
int eta = 16;  
string etaStr = to_string(eta);  
cout << "Hai " + etaStr + " anni." << endl;
```

## Convertire stringhe in numeri

Per trasformare una stringa che contiene un numero in un valore numerico:

```
string numStr = "42";  
int num = stoi(numStr);    // da stringa a int  
cout << num + 1 << endl;   // 43  
  
string decimaleStr = "3.14";  
double dec = stod(decimaleStr); // da stringa a double
```

## Leggere stringhe con spazi

`cin >>` si ferma al primo spazio. Per leggere tutta la riga usa `getline()` :

```
string nome;
cout << "Inserisci il tuo nome completo: ";
getline(cin, nome);
cout << "Ciao, " << nome << "!" << endl;
```

## Altri metodi utili

```
// Controlla se la stringa è vuota
string vuota = "";
cout << vuota.empty() << endl;    // 1 (true)

// Trova la posizione di un carattere
string nome = "Alice";
cout << nome.find('l') << endl;    // 1

// Sostituisce una porzione
string s = "Ciao, mondo!";
s.replace(6, 5, "Alice");           // sostituisce 5 caratteri dalla
// posizione 6
cout << s << endl;                 // Ciao, Alice!

// Inserisce del testo in una posizione
string str = "Ciao!";
str.insert(4, ", mondo");
cout << str << endl;              // Ciao, mondo!
```

## Esempio pratico

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string nome, cognome;

    cout << "Inserisci il nome: ";
    cin >> nome;
    cout << "Inserisci il cognome: ";
    cin >> cognome;

    string nomeCompleto = nome + " " + cognome;

    cout << endl;
    cout << "Nome completo: " << nomeCompleto << endl;
    cout << "Lunghezza: " << nomeCompleto.length() << " caratteri" << endl;
    cout << "Iniziale nome: " << nome[0] << endl;
    cout << "Iniziale cognome: " << cognome[0] << endl;

    // Conta le vocali nel nome completo
    string vocali = "aeiouAEIOU";
    int contaVocali = 0;
    for (char c : nomeCompleto) {
        if (vocali.find(c) != string::npos) {
            contaVocali++;
        }
    }
    cout << "Numero di vocali: " << contaVocali << endl;

    return 0;
}
```